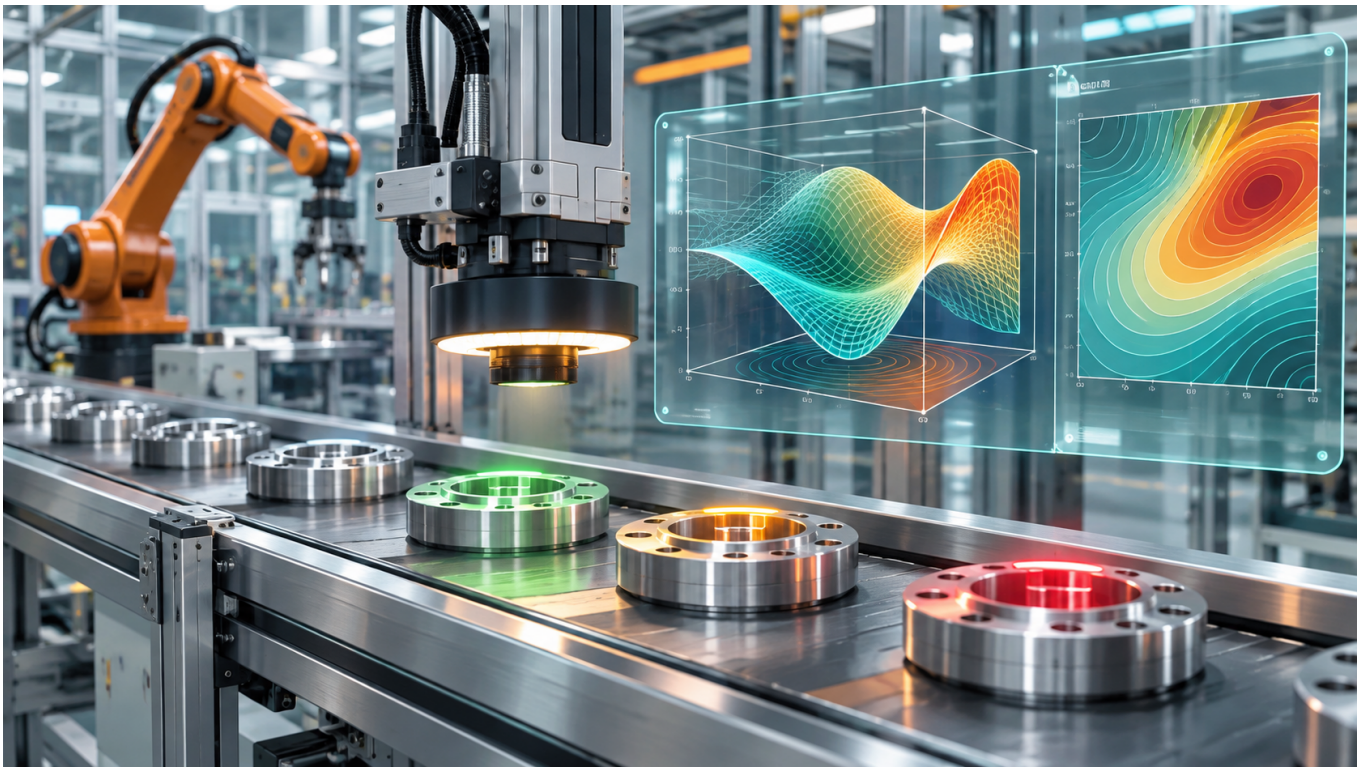
SCIENTIFIC RESEARCH REPORT | WP_027

Mathematical Quality and Scrap Scores

Reusable mathematical execution graphs for industrial process-quality monitoring and controlled inspection routing



Verified research outcome

50 versioned score definitions; 51 retained paceval computation graphs; 10,000 deterministic synthetic observations; 13,843 scalar evaluations; 1,600 batch checks; 3,721 3D surface values. Independent numerical samples passed the declared audit. No real-plant yield or calibrated scrap-risk claim is made.

Cover image: conceptual rendering of automated inspection. Scientific figures in the following pages use actual paceval-generated numerical datasets.

1. Industrial use case and state of the art

Industrial quality engineering seeks to detect nonconforming manufacturing conditions before material progresses downstream. In an automated machining or semiconductor process, temperature, pressure, speed, vibration, and wear may vary across time and products. Engineers combine statistical process control, product specifications, sensor validity checks, predictive models, and inspection policies. A mathematical quality score can organize the evaluation, but cannot by itself certify quality or identify physical defects.

Concrete application scenario

A hypothetical manufacturing cell continuously produces five normalized sensor deviations. For each item, a registered score rule generates a dimensionless risk proxy. An independent host policy routes a score of at least 0.60 to additional inspection. The inspection flag is not an automatic scrap decision. Technical validation, safety permissions, and product conformance decisions remain external to the formula engine.

Research context and alternatives

The UCI SECOM collection is an example of manufacturing pass/fail records with hundreds of sensor variables and missing observations. AI4I 2020 contains 10,000 deliberately synthetic machine-operating records and machine-failure labels, not manufactured-part scrap labels. Neither external dataset was used to generate the numerical results in this paper. Industry-grade use requires leakage-resistant validation, measurement-system analysis, uncertainty treatment, class-imbalance handling and observed quality outcomes.

Research question

Can fifty different, explicitly versioned mathematical scoring rules be executed and repeatedly reused through persistent paceval computation graphs, integrated with an inspection-routing host, and subjected to independent numerical verification using deterministic synthetic inputs?

The study answers that bounded execution question. It does not assess manufacturing model calibration or real scrap-reduction effectiveness.

2. Mathematical definition and model scope

Let t, p, s, v, w denote dimensionless standardized process deviations for temperature, pressure, rotational speed, vibration, and tool wear. The explicit demonstration input contract is [-2,2] per component; the generated records occupy [-1.6,1.6]. For each registered model, a nonlinear latent score is transformed by a logistic function.

Common logistic transformation

$S = 1 / (1 + \exp(-z))$. The output lies between 0 and 1 for finite valid inputs. That range alone does not establish probabilistic calibration.

Five family-level nonlinear expressions

Family 1: $z = a*t + b*p + c*s + 0.35*v + 0.25*w$.

Family 2: $z = a*t^2 + b*p^2 + c*s^2 + 0.30*v - 0.70$.

Family 3: $z = a*t*p + b*s*v + c*w + 0.12*t^2 - 0.15$.

Family 4: $z = a*\sin(t) + b*\cos(p) + c*s*w + 0.30*v$.

Family 5: $z = a*\ln(1+t^2) + b*\mathrm{sqrt}(1+p^2) + c*v*w + 0.15*s - 0.45$.

Ten declared coefficient choices (a,b,c) per family yield fifty distinct expressions. A separately registered version of family 1 modifies coefficient a without replacing the previous paceval handle. Exact expression strings and coefficient values are published in examples/score_registry.csv.

Decision function is owned by the C++ host

For a score S, inspect(S) = 1 when S is at least 0.60, and 0 otherwise. This value is an illustrative process-routing threshold, not an optimized or validated industrial release criterion. The host validates input ranges, manages synthetic labels, routes decisions, and writes the audit records.

3. paceval execution architecture

The original paceval_main.h and paceval_main.cpp facade is built with C++17 and linked to a separately supplied compatible Linux runtime. No substitute mathematical engine is included. Graphs are registered once and reused across multiple input vectors.

Verifiable software boundaries

Persistent graph creation	**paceval_CreateComputation**
Scalar mathematical evaluation	**paceval_dGetComputationResult**
Packed batch evaluation	**paceval_dGetComputationResultExt**
Graph destruction	**paceval_DeleteComputation**
Library lifecycle	**InitializeLibrary / FreeLibrary**

Controlled change management

A 51st computation graph is created from a changed coefficient of family 1. The original graph remains present and callable. This demonstrates the separation between changing an input value and creating a new expression version. Formula strings are externally inspectable rather than silently replaced.

Independent application-layer controls

C++ generates deterministic records using a fixed integer seed, checks finite normalized input domains, selects the registered model, applies the inspection threshold, and records the synthetic observation and label. Pointwise evaluation is not a proof of a score model over every possible input, and no paceval interval certification is asserted.

4. Executed deterministic workload

The executed experiments use 10,000 synthetic five-feature observations and fifty independently declared formulas. An additional 61-point comparison of two persistent formula versions demonstrates update traceability. An independently generated synthetic defect-label mechanism is not trained on any score model.

Distinct mathematical score definitions	**50**
Persistent graphs including formula version 2	**51**
Synthetic observations	**10,000**
Scalar paceval mathematical evaluations	**13,843**
Verified packed batch results	**1,600**
Three-dimensional surface values	**3,721**
Independent 80-digit reference checks	**500 / 500 PASS**
SDK integration CTest	**1 / 1 PASS**

Published research evidence

The accompanying machine-readable files preserve registry formulas, input features, scores, routing decisions, surface points, model-version trajectories, SDK execution records, and independent audit reports. The samples are a deliberately limited mathematical execution benchmark and do not represent measured manufacturing yield.

5. Inspection decisions and score families

Across 10,000 synthetic observations, 2,046 exceed the demonstration threshold and are routed to inspection. Separately, 3,710 observations have a synthetic positive defect label created using an independent data-generation mechanism. These are not industrial production quantities.

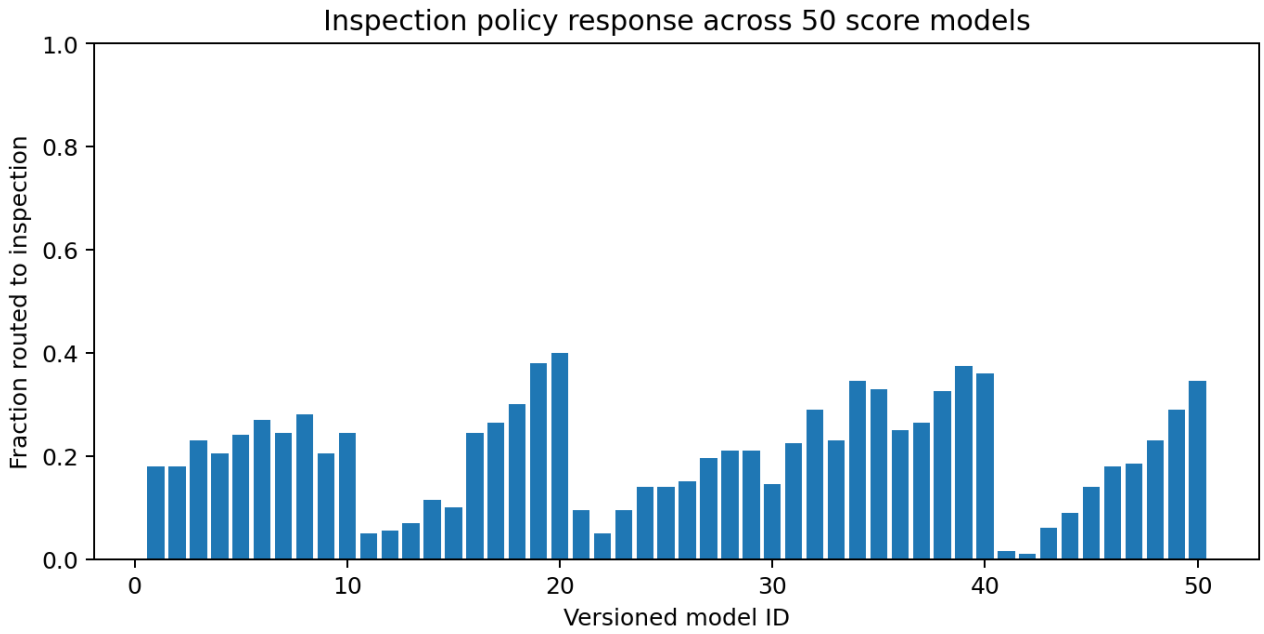


Figure 3. Fraction of synthetic records flagged for inspection for each of fifty registered paceval score formulas.

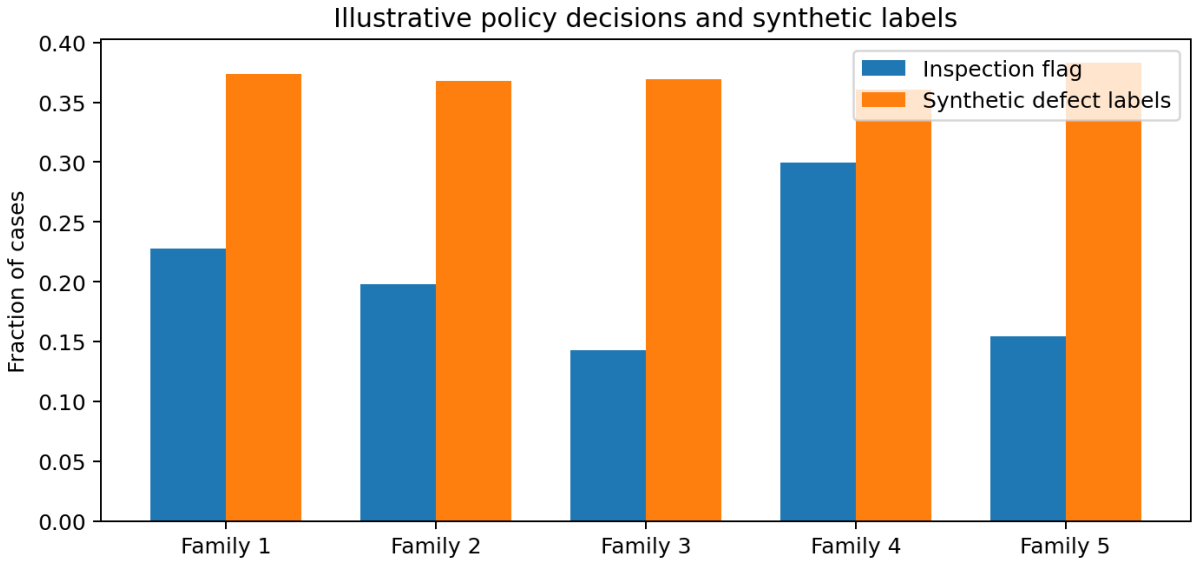


Figure 4. Inspection decisions by formula family and the independently generated synthetic labels.

6. Process-score distribution

The distribution of computed scores and the threshold-based routing profile illustrate how different mathematical forms lead to different inspection workloads. No inference about actual part quality is justified without production-labeled calibration and validation.

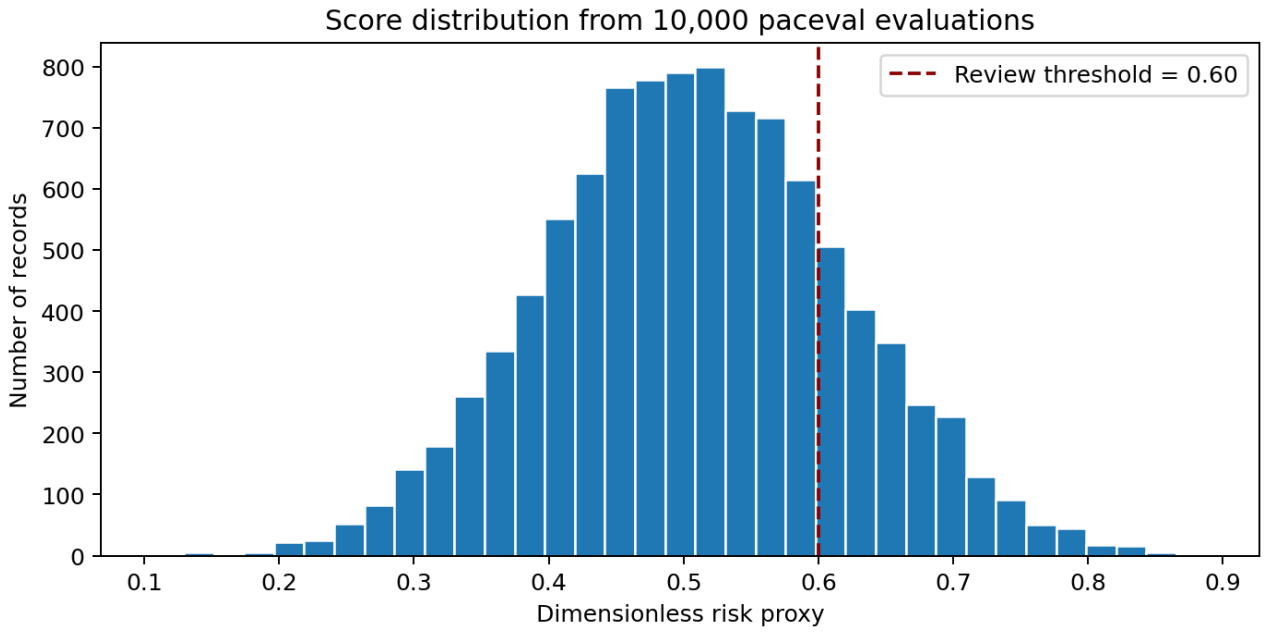


Figure 2. Distribution from 10,000 actual paceval score outputs; the dashed line is the declared inspection threshold.

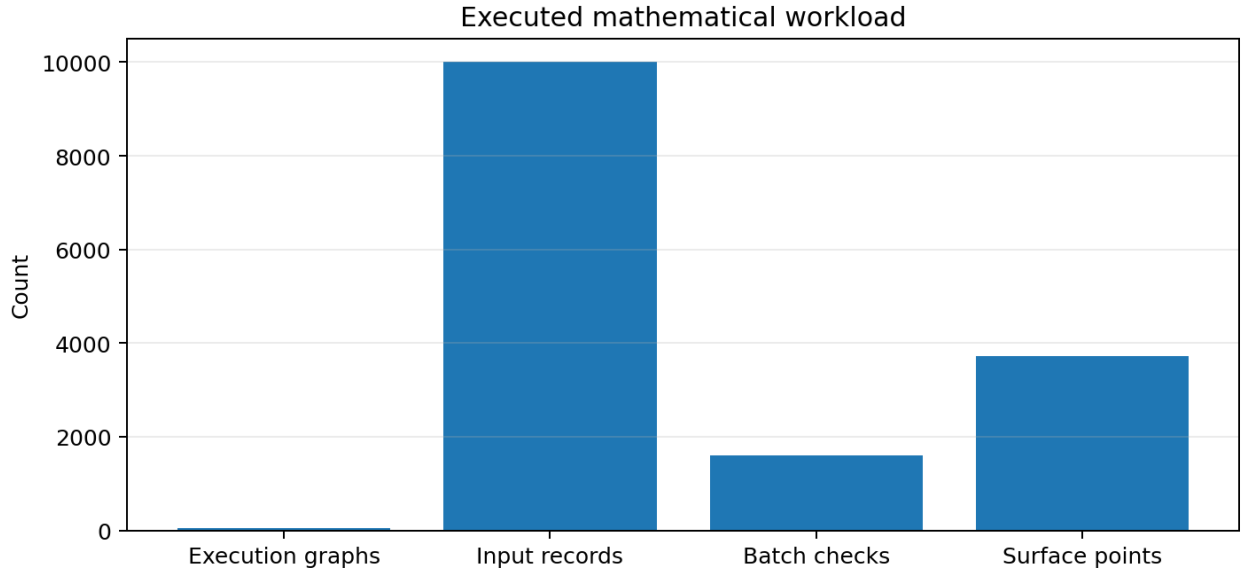


Figure 1. Numbers of executed graphs, process observations, tested packed evaluations, and surface samples.

7. Three-dimensional quality-score surface

A fixed version of formula family 1 is evaluated on a 61-by-61 grid of temperature and pressure deviations with other inputs fixed. The surface is constructed from 3,721 actual paceval mathematical results, not a generated numerical illustration.

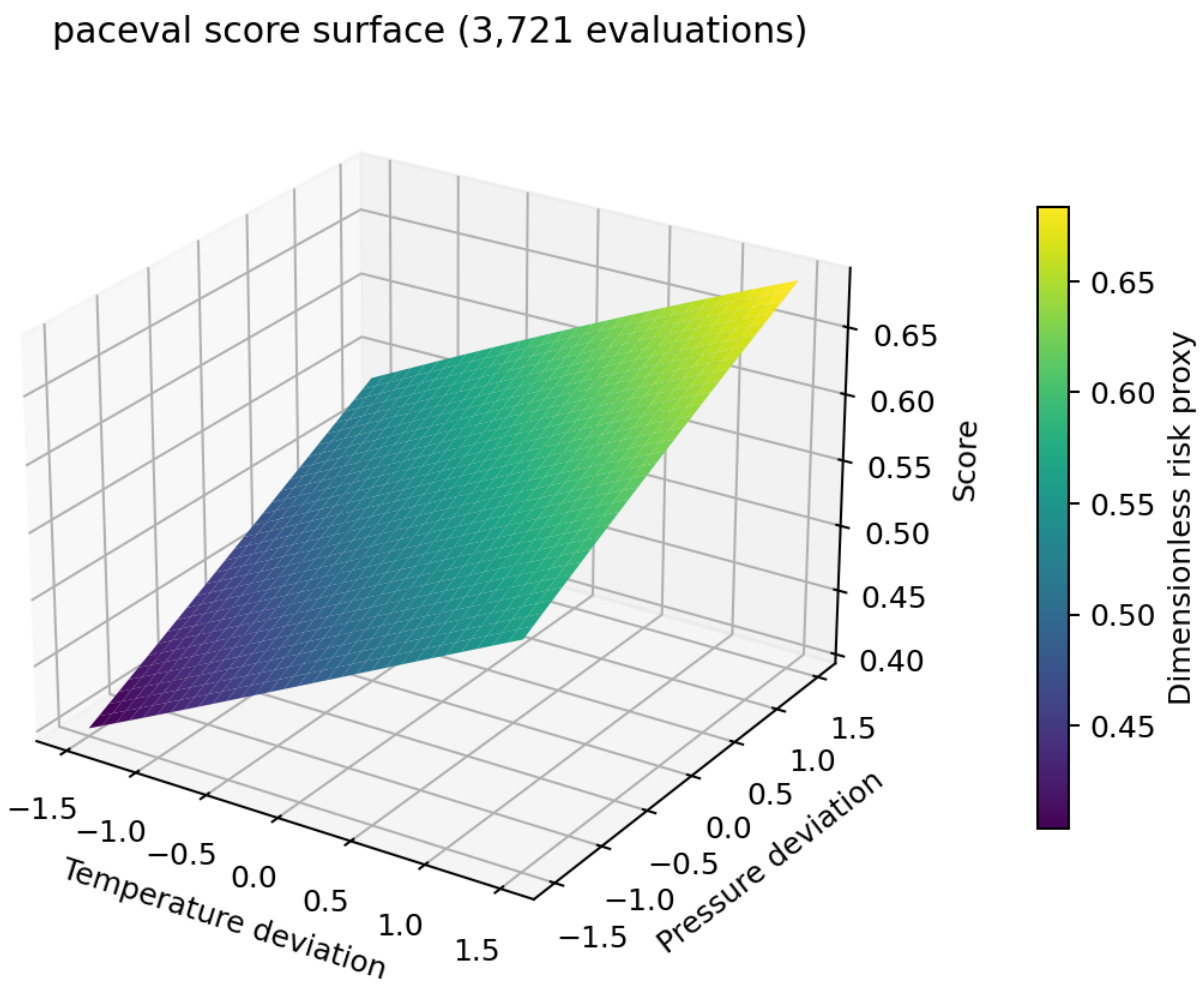


Figure 5. Reusable logistic response surface for two normalized process features, evaluated through paceval.

The plot is a sensitivity illustration for an uncalibrated synthetic model; it is not a manufacturing process-window specification.

8. Contours and formula revision

The contour map projects the same paceval score surface into two dimensions. Such graphics could help engineers inspect the internal behaviour of a controlled formula before it is considered for a product-specific quality workflow.

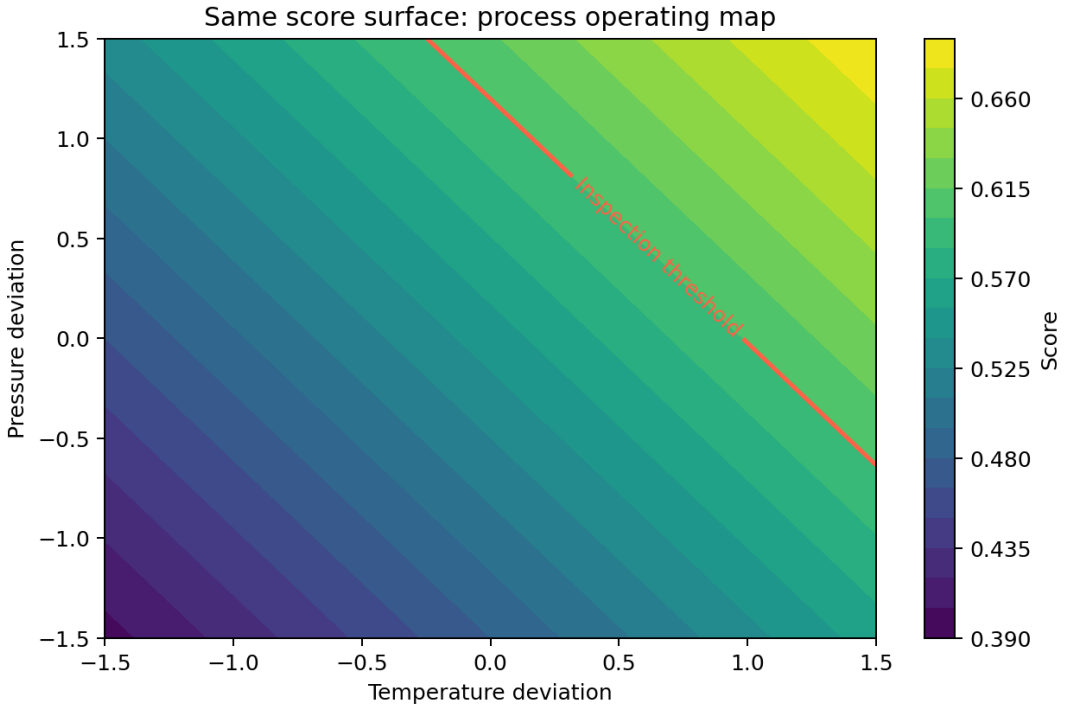


Figure 6. Process-operating contour map of the evaluated mathematical surface.

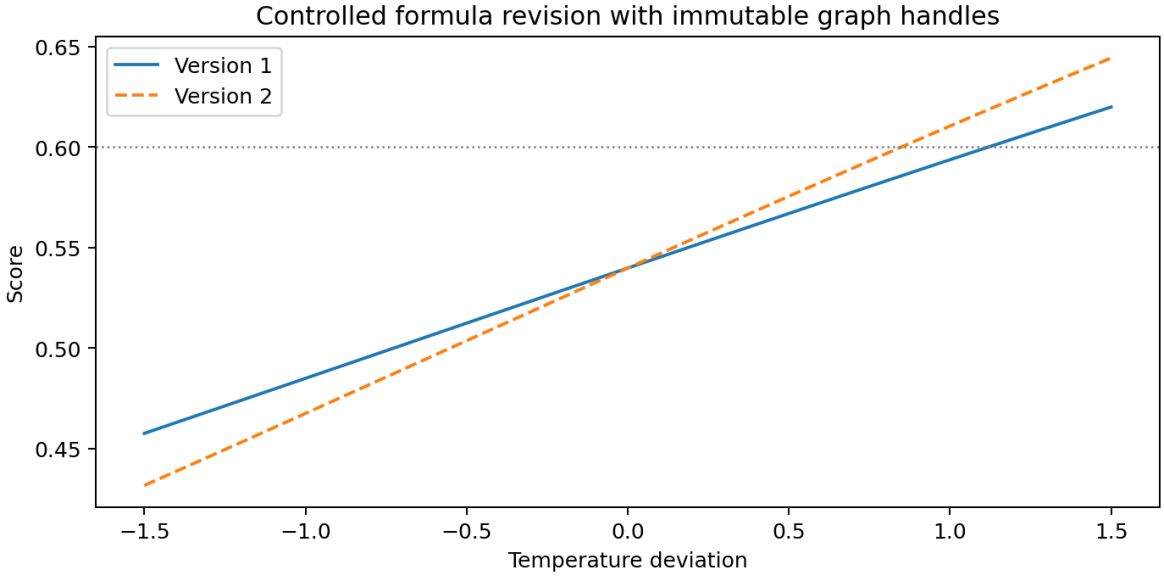


Figure 7. Immutable original graph and a separately registered second formula version.

9. Interpretability and limitations

The synthetic label-generation function differs from the fifty tested score models. Consequently, diagnostic agreement between scores and synthetic labels must not be interpreted as a fitted model performance result. A score bounded by zero and one is not automatically a probability.

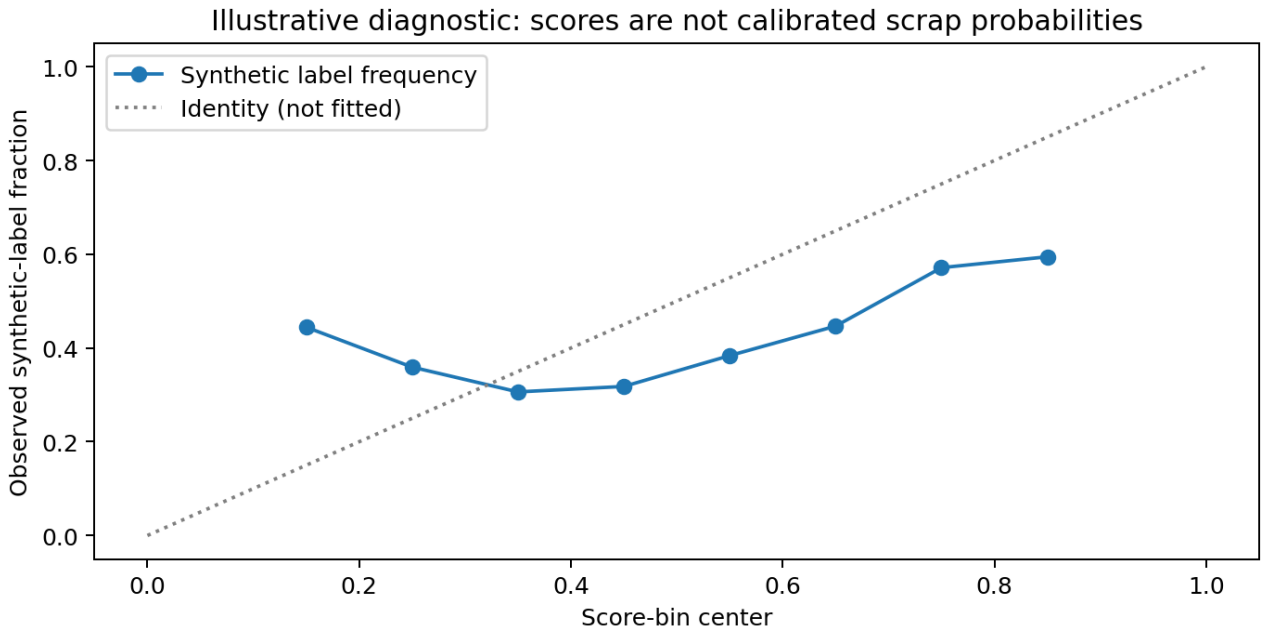


Figure 8. Exploratory label frequency by score bin. The diagnostic explicitly does not establish calibration.

Scope of the research conclusion

Successfully demonstrated: mathematical graph construction, repeated evaluation of five nonlinear families, persistent expression versioning, packed batch calls, auditable decisions, reproducible CSV exports, and an independent numerical audit. Not established: predictive manufacturing quality, measurement traceability, site-specific scrap reduction, financial savings, robust handling of missing real sensors, or production certification.

10. Reproducibility and external references

Build and verification

Build from src/CMakeLists.txt with the C++17 compiler, CMake and an externally provided ABI-compatible paceval Linux shared library; run the quality_scores executable with --out; execute CTest. The project includes Python verification scripts, a high-precision reference, figure rendering, and a README mathematics and local-link checker.

Included project evidence

The project-root README provides exact commands and relative links. Data: examples/score_registry.csv, quality_records.csv, quality_surface.csv, score_summary.csv and formula_versions.csv. Audits: results/sdk_execution.txt, ctest_sdk.txt, decimal_audit.json and result_audit.json. Graphics: eight numbered pairs of PNG and vector-PDF files.

Selected scientific and technical references

1. UCI Machine Learning Repository. SECOM manufacturing dataset. https://archive.ics.uci.edu/dataset/179/secom

2. UCI Machine Learning Repository. AI4I 2020 Predictive Maintenance Dataset, DOI: 10.24432/C5HS5C. Synthetic machine-failure observations; CC BY 4.0.

3. NIST Engineering Laboratory, manufacturing measurement and process engineering context. https://www.nist.gov/el

4. paceval Mathematical Engine C/C++ API documentation. https://paceval.com/api/

Conclusion

The executed synthetic research demonstrator confirms that reusable paceval mathematical graphs can power a versioned catalogue of nonlinear manufacturing quality scores in a controlled C++ application. The work substantiates mathematical execution functionality while preserving a clear boundary between numeric scoring and industrial quality validation.